

Correction étudiant — TD/TP 2

Du C au C++ moderne
D'une API C à une première interface C++
MAM4 — Programmation C++

Les programmes complets accompagnent cette correction. Compilation :

```
g++ -std=c++20 -Wall -Wextra -pedantic \
    -fsanitize=address,undefined -g \
    int_array.cpp main.cpp -o int_array
```

Étape 0 — Comprendre le tableau C

Champ	Signification	Règle principale
capacity	places disponibles	capacity >= 0
size	éléments utilisés	0 <= size <= capacity
data	adresse de la zone	cohérente avec capacity

`create_array`, `push_back` et `destroy_array` modifient la structure. `at` et `sum` la lisent. L'appelant doit appeler `destroy_array` exactement une fois : un oubli produit une fuite ; deux libérations de la même adresse sont incorrectes. Lorsque le tableau est plein, sa capacité double.

Le code C est refusé tel quel par g++, notamment parce que `malloc` et `realloc` renvoient un `void*`.

À retenir. Un invariant est une règle qui doit toujours rester vraie pour que l'objet soit correct.

Étape 1 — Première version C++ à fonctions libres

```
struct IntArray {
    int capacity{};
    int size{};
    int* data{nullptr};
};

void create_array(IntArray& array, int initial_capacity);
void push_back(IntArray& array, int value);
int at(const IntArray& array, int index);
int sum(const IntArray& array);
void fill(IntArray& array, int value);
void destroy_array(IntArray& array);
```

Les opérations qui modifient reçoivent `IntArray&`. Celles qui observent reçoivent `const IntArray&`.

```
void push_back(IntArray& array, int value)
{
    if (array.size == array.capacity) {
        const int new_capacity{2 * array.capacity};
        int* new_data{new int[new_capacity]{};

        for (int i{0}; i < array.size; ++i) {
            new_data[i] = array.data[i];
        }

        delete[] array.data;
        array.data = new_data;
        array.capacity = new_capacity;
    }
    array.data[array.size] = value;
    ++array.size;
}
```

Pour agrandir : créer la nouvelle zone, copier les valeurs, supprimer l'ancienne zone, puis installer la nouvelle.

Étape 2 — Lire ou modifier

```
int at(const IntArray& array, int index)
{
    if (index < 0 || index >= array.size) {
        throw std::out_of_range{"IntArray index out of range"};
    }
    return array.data[index];
}

int sum(const IntArray& array)
{
    int result{};
    for (int i{0}; i < array.size; ++i) {
        result += array.data[i];
    }
    return result;
}

void fill(IntArray& array, int value)
{
    for (int i{0}; i < array.size; ++i) {
        array.data[i] = value;
    }
}
```

La ligne `array.size = 0;` est refusée dans `sum`, car `array` est observé à travers une référence constante.

Étape 3 — Surcharger `push_back`

```
void push_back(IntArray& array, int value, int repetitions)
{
    if (repetitions < 0) {
        throw std::invalid_argument{
            "repetitions must be non-negative"
        };
    }
    for (int i{0}; i < repetitions; ++i) {
        push_back(array, value);
    }
}
```

`push_back(values, 7)` sélectionne la première version; `push_back(values, 7, 3)` sélectionne la seconde. Celle-ci réutilise la première : l'agrandissement n'est écrit qu'une fois.

Étape 4 — Passer aux fonctions membres

```
struct IntArray {
    int capacity{};
    int size{};
    int* data{nullptr};

    void push_back(int value);
    void push_back(int value, int repetitions);
    int at(int index) const;
    int sum() const;
    void fill(int value);
};
```

```
int IntArray::sum() const
{
    int result{};
    for (int i{0}; i < size; ++i) {
        result += data[i];
    }
    return result;
}

void IntArray::push_back(int value, int repetitions)
{
    if (repetitions < 0) {
        throw std::invalid_argument{
            "repetitions must be non-negative"
        };
    }
    for (int i{0}; i < repetitions; ++i) {
        push_back(value);
    }
}
```

Les appels sont désormais `values.push_back(7)`, `values.at(0)`, `values.sum()` et `values.fill(5)`. Le `const` final signifie que la fonction membre n'altère pas l'objet.

À retenir. Une fonction membre rapproche l'opération des données, mais ne protège pas encore les champs publics.

Étape 5 — Tests essentiels

```
IntArray values;
create_array(values, 2);

values.push_back(10);
values.push_back(20);
assert(values.capacity == 2);

values.push_back(30);
assert(values.size == 3);
assert(values.capacity == 4);
assert(values.at(0) == 10);
assert(values.at(2) == 30);

values.push_back(40, 6);
assert(values.size == 9);
assert(values.capacity == 16);

values.fill(7);
assert(values.sum() == 63);

destroy_array(values);
assert(values.size == 0);
assert(values.capacity == 0);
assert(values.data == nullptr);
```

Les indices invalides `-1` et `values.size` doivent chacun produire `std::out_of_range`. Les sanitizers doivent rester silencieux pendant les tests.

Étape 6 — Les deux problèmes qui restent

1. `values.size = -1`; compile parce que les champs sont publics ; cette ligne casse l'invariant.
2. Des données privées permettront d'obliger le code extérieur à passer par une interface contrôlée : c'est l'encapsulation.
3. Un constructeur créera directement un objet correct.
4. Un destructeur libérera automatiquement sa mémoire.
5. `IntArray b = a`; recopie l'adresse contenue dans `data` : les deux objets désignent la même zone.

À retenir. Copier un pointeur ne copie pas les données pointées. Les fonctions membres ne suffisent pas encore : l'état reste public et la mémoire reste manuelle.

Bilan

Notion réutilisée ou introduite	Rôle dans <code>IntArray</code>
<code>T&</code>	modifier le tableau fourni
<code>const T&</code>	observer sans copier ni modifier
surcharge	proposer deux formes de <code>push_back</code>
fonction membre	appeler une opération sur l'objet concerné
invariant	exprimer les règles d'un tableau correct
sanitizers	détecter des erreurs mémoire pendant les tests

Le code complet se trouve dans les dossiers `etape3_fonctions_libres` et `etape4_fonctions_membres` distribués avec cette correction.